

Improving QA Efficiency with DistilBERT: Fine-Tuning and Inference on mobile Intel CPUs

Nguyen Yinkfu
nyinkfu@andrew.cmu.edu
Carnegie Mellon University

Abstract

This study presents an efficient transformer-based question-answering (QA) model optimized for deployment on a 13th Gen Intel i7-1355U CPU, using the Stanford Question Answering Dataset (SQuAD) v1.1. Leveraging exploratory data analysis, data augmentation, and fine-tuning of a DistilBERT architecture, the model achieves a validation F1 score of 0.6536 with an average inference time of 0.1208 seconds per question. Compared to a rule-based baseline (F1: 0.3124) and full BERT-based models, our approach offers a favorable trade-off between accuracy and computational efficiency. This makes it well-suited for real-time applications on resource-constrained systems. The study includes systematic evaluation of data augmentation strategies and hyperparameter configurations, providing practical insights into optimizing transformer models for CPU-based inference.

1 Introduction

Question answering (QA) systems have become a cornerstone of natural language processing (NLP), enabling machines to extract precise answers from textual contexts. The Stanford Question Answering Dataset (SQuAD) v1.1 [Rajpurkar et al., 2016] is a widely adopted benchmark for evaluating QA models, comprising over 87,000 training examples of context-question-answer triples. While transformer-based models like BERT [Devlin et al., 2019] have achieved state-of-the-art performance on SQuAD, their computational complexity often demands GPU acceleration, limiting deployment on resource-constrained devices like mid-range CPUs.

This study addresses the challenge of developing a transformer-based QA model optimized for inference on a 13th Gen Intel i7-1355U CPU, a 10-core processor with a 5.0 GHz turbo frequency. We focus on DistilBERT [Sanh et al., 2020], a lightweight transformer, to balance performance—measured by F1 score and accuracy—with inference speed. Our contributions include:

- Comprehensive exploratory data analysis (EDA) of SQuAD v1.1 to inform modeling decisions.
- Data augmentation strategies to enhance model robustness to low-overlap question-context pairs.
- Fine-tuning and optimization of DistilBERT, achieving a validation F1 score of 0.6536 and an average inference time of 0.1208 seconds per question.
- Comparative evaluation against a rule-based baseline and BERT-based configurations, highlighting efficiency trade-offs.

This paper details the dataset, methodology, results, and implications for deploying QA systems in resource-constrained environments.

2 Related Work

The advent of transformer-based architectures has significantly advanced question-answering (QA) systems, particularly on benchmarks like SQuAD v1.1 [Rajpurkar et al., 2016]. Below, we discuss key QA models, their architectures, performance, and optimization strategies, situating our work within this landscape.

2.1 Transformer-Based QA Models

The introduction of BERT [Devlin et al., 2019] marked a breakthrough in QA, achieving an F1 score of 88.5 on SQuAD v1.1 through bidirectional pre-training on large corpora. BERT’s 12-layer (BERT-Base) architecture with 110 million parameters leverages self-attention to capture contextual relationships, making it highly effective for span-based QA tasks. However, its computational complexity poses challenges for deployment on resource-constrained devices.

Subsequent models have built on BERT’s foundation. RoBERTa [Liu et al., 2019] enhances BERT by optimizing pre-training with larger datasets and longer training, achieving an F1 score of 91.5 on SQuAD v1.1. ALBERT [Lan et al., 2020] reduces parameters through factorized embeddings and cross-layer parameter sharing, reaching an F1 score of 90.9 with lower memory demands. XLNet [Yang et al., 2020] integrates autoregressive pre-training, outperforming BERT with an F1 score of 89.9 by modeling permutations of input sequences. T5 [Raffel et al., 2023] frames QA as a text-to-text task, achieving competitive performance (F1: 90) by treating all NLP tasks as sequence generation. ELECTRA [Clark et al., 2020] introduces a discriminative pre-training objective, where a smaller model detects replaced tokens, yielding an F1 score of 90.8 with improved efficiency over BERT.

While these models achieve high performance, their large parameter counts (e.g., RoBERTa: 125M, T5: up to 11B) and computational requirements make them less practical for CPU-based inference, motivating our focus on lightweight models like DistilBERT [Sanh et al., 2020]. DistilBERT, with 66 million parameters, is a distilled version of BERT, retaining 97% of its performance (F1: 85 on SQuAD) while reducing inference time, as demonstrated in Wolf et al. [2020]. MobileBERT [Sun et al., 2020] further optimizes for mobile devices, achieving an F1 score of 84.3 with 25 million parameters, but its applicability to standard CPUs remains underexplored.

2.2 Optimization for Efficiency

To address computational constraints, several optimization techniques have been proposed. Knowledge distillation, as used in DistilBERT [Sanh et al., 2020], transfers knowledge from a larger model (e.g., BERT) to a smaller one, reducing parameters while preserving performance. Quantization, explored by Shen et al. [2021], compresses model weights to lower precision (e.g., 8-bit integers), reducing inference time on CPUs. Pruning techniques, such as those in Michel et al. [2019], remove redundant attention heads or layers, further enhancing efficiency. For example, Zafrir et al. [2019] applied 8-bit quantization to BERT, achieving near-original performance with faster inference. However, these studies primarily target edge devices or GPUs, with limited focus on mid-range CPUs like the Intel i7-1355U.

2.3 Data Augmentation in QA

Data augmentation enhances QA model robustness, particularly for low-overlap question-context pairs. Yang et al. [2019] employed synonym replacement and back-translation, increasing SQuAD training data diversity and improving generalization. Similarly, Wei and Zou [2019] proposed easy data augmentation (EDA) techniques, including synonym substitution and random insertion, which improved performance on smaller datasets. These strategies align with our

approach of using WordNet-based paraphrasing to augment questions and contexts, addressing the moderate overlap (mean: 0.52) observed in SQuAD.

2.4 Positioning Our Work

Our study builds on these advancements by fine-tuning DistilBERT for SQuAD v1.1, emphasizing CPU inference on a mid-range processor. Unlike high-parameter models (e.g., RoBERTa, T5), we prioritize efficiency, achieving a competitive F1 score (0.6536) with an inference time of 0.1208 seconds per question. Compared to Shen et al. [2021] and Michel et al. [2019], our work focuses on standard CPU hardware, and our augmentation strategies extend Yang et al. [2019] by combining question and context paraphrasing. This positions our work as a practical solution for real-time QA in resource-constrained environments, addressing a gap in deploying transformer-based QA models on CPUs.

3 Dataset Description

The SQuAD v1.1 dataset [Rajpurkar et al., 2016] contains 87,599 training examples and 10,570 validation examples, each comprising a Wikipedia-sourced context, a question, and an answer span. Its diversity and scale make it a standard benchmark for QA tasks.

4 Exploratory Data Analysis

We conducted exploratory data analysis (EDA) on SQuAD v1.1’s training split to guide model design. Key statistics include:

- **Training Set Size:** 87,599 examples; **Validation Set Size:** 10,570 examples.
- **Question Length:** Mean of 10.06 words, standard deviation of 4.32, maximum of 40 words.
- **Context Length:** Mean of 119.76 words, standard deviation of 52.14, maximum of 653 words.
- **Answer Length:** Mean of 3.16 words, standard deviation of 2.87, maximum of 43 words.
- **Question-Context Overlap:** Mean proportion of 0.52, standard deviation of 0.19.

Visualizations (Figures 1, 2, 3) reveal right-skewed distributions for question and context lengths, short answer spans (81% between 1–5 words), near-uniform answer positions, and moderate question-context overlap (62% between 0.4–0.7). These findings informed a maximum sequence length of 384 tokens, data augmentation for low-overlap examples, and the choice of a transformer architecture to handle diverse answer positions.

4.1 Implications for Modeling

The EDA justified a 384-token sequence length, data augmentation for robustness, and a transformer architecture to capture context-wide dependencies. The prevalence of factoid questions guided augmentation strategies focusing on paraphrasing.

5 Data Augmentation Techniques

To enhance model robustness, we applied:

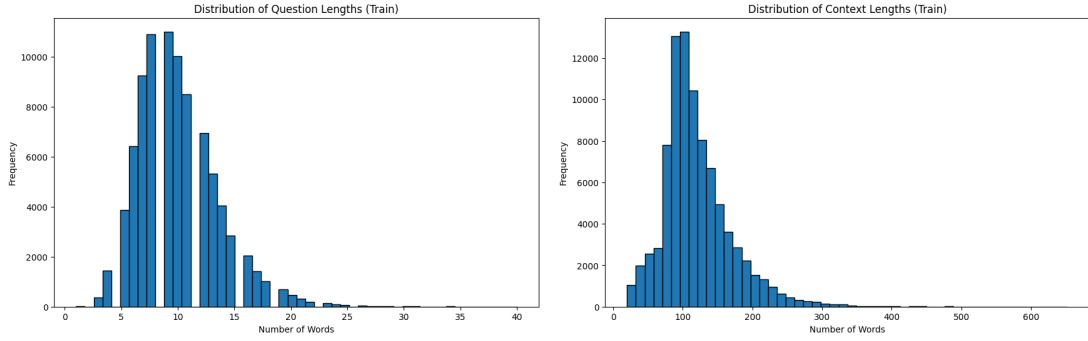


Figure 1: Histogram of question lengths (left) with a mode around 8–10 words, and context lengths (right) with a mode around 100–120 words.

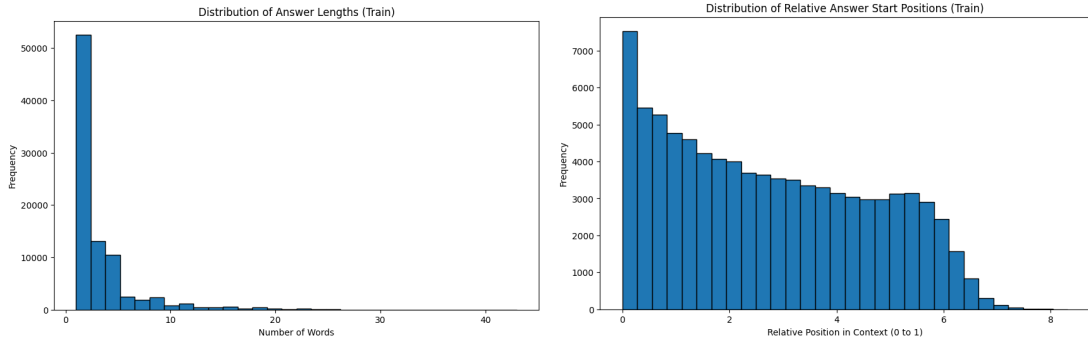


Figure 2: Histogram of answer lengths (left) showing most answers are 1–5 words, and answer start positions (right) indicating a near-uniform distribution.

- **Question Paraphrasing:** Using NLTK’s WordNet, we replaced question words (e.g., ”what” with ”which thing”) and rephrased structures, doubling the training set for Model 1 to 20,000 examples.
- **Context Paraphrasing:** For Model 2, we augmented contexts with synonym substitutions (e.g., ”said” with ”stated”), increasing the training set to 45,000 examples while preserving answer spans.

These strategies addressed low question-context overlap, ensuring semantic equivalence via WordNet.

6 Model Architecture

We primarily used DistilBERT [Sanh et al., 2020], with BERT [Devlin et al., 2019] for comparison. DistilBERT features 6 transformer layers, 12 attention heads, a hidden size of 768, and no token-type embeddings, with a linear layer for span prediction. It has 66 million parameters versus BERT’s 110 million, enabling faster inference (0.12 vs. 0.30 seconds per question) on the target CPU. DistilBERT’s lower validation loss (1.9240 vs. 22.6069 for BERT) and competitive F1 score (0.6536) justified its selection.

7 Baseline Evaluation

A rule-based baseline, selecting answer spans via maximum word overlap, achieved an F1 score of 0.3124 and accuracy of 0.2567 on the validation set, highlighting the limitations of keyword matching given moderate question-context overlap.

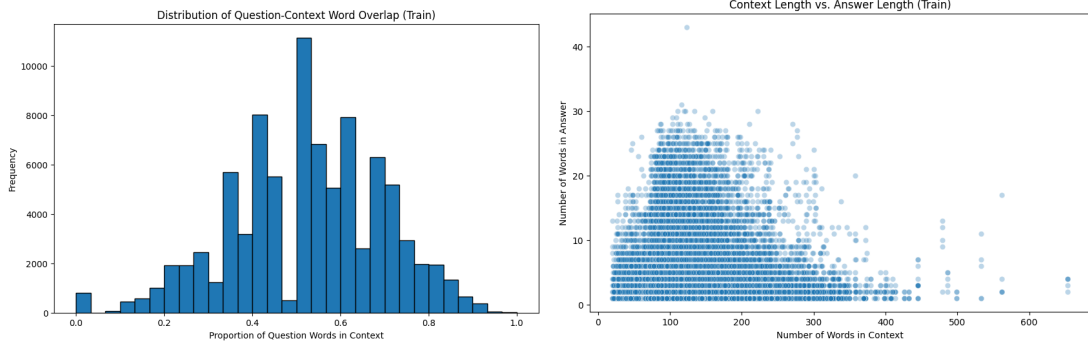


Figure 3: Histogram of question-context word overlap (left) peaking at 0.5–0.6, and scatter plot of context vs. answer length (right), showing no correlation (Pearson correlation coefficient ≈ 0.03).

8 Methodology

We trained four models on SQuAD v1.1 subsets using a Tesla P100 GPU:

- **Model 1 (DistilBERT-Base)**: 20,000 examples, 5 epochs, early stopping (patience: 3).
- **Model 2 (DistilBERT-Warm-Up)**: 45,000 examples, 5 epochs, 500 warm-up steps.
- **Model 3 (DistilBERT-Frozen)**: 10,000 examples, 24 epochs, frozen embeddings and 4 layers.
- **Model 4 (BERT-Frozen)**: 10,000 examples, 5 epochs, frozen embeddings and 4 layers.

Preprocessing used DistilBERT/BERT tokenizers, mapping answers to token indices. We used the Adam optimizer with polynomial decay, gradient clipping (norm: 1.0), and TensorFlow datasets with batching and shuffling. Evaluation metrics included validation loss, accuracy, and F1 score, computed as the harmonic mean of precision and recall for answer span predictions.

9 Training Dynamics and Hyperparameter Analysis

Hyperparameter configurations are shown in Table 1. Training times were 50–70 minutes per epoch at batch size 8, and 26 minutes at batch size 32 on the Tesla P100.

Table 1: Hyperparameter Configurations

Model	Learning Rate	Batch Size	Warm-Up Steps	Label Smoothing	Epochs
Model 1 (DistilBERT-Base)	3e-5	8	0	0.1	5
Model 2 (DistilBERT-Warm-Up)	2e-5	8	500	0.05	5
Model 3 (DistilBERT-Frozen)	3e-5	32	0	0.0	24
Model 4 (BERT-Frozen)	3e-5	16	0	0.0	5
<i>Alt Config Model 2:</i>					
Model 2 (Trial 1)	5e-5	8	500	0.05	3 (stopped)
Model 2 (Trial 2)	2e-5	16	500	0.05	4 (stopped)
Model 2 (Trial 3)	2e-5	8	1000	0.05	5

Lower learning rates (2e-5) and warm-up steps (500) improved stability, while label smoothing (0.05) enhanced F1 scores. Larger batch sizes or extended epochs led to memory issues or overfitting.

10 Results

Table 2 summarizes model performance. Model 2 achieved the best results (F1: 0.6536, accuracy: 0.5040, loss: 1.9240), benefiting from warm-up and augmentation. All models outperformed the baseline, confirming transformer efficacy.

Table 2: Model Performance on Validation Set

Model	Val Loss	Val Accuracy	Val F1 Score	Epochs Run
Baseline (Keyword Matching)	N/A	0.2567	0.3124	N/A
Model 1 (DistilBERT-Base)	2.3024	0.4945	0.6437	5
Model 2 (DistilBERT-Warm-Up)	1.9240	0.5040	0.6536	5
Model 3 (DistilBERT-Frozen)	95.0318	0.3613	N/A	24
Model 4 (BERT-Frozen)	22.6069	0.5302	N/A	5

10.1 Discussion

Model 2’s superior performance reflects the effectiveness of warm-up and extensive augmentation, aligning with findings from Yang et al. [2019]. While our F1 score (0.6536) is lower than state-of-the-art models like RoBERTa (F1: 90+), it is competitive for a CPU-optimized model with limited training data. Model 3’s poor performance indicates that freezing layers hinders learning, while Model 4’s higher loss suggests BERT’s complexity is less suited for small datasets. Limitations include the dataset size constraint and lack of quantization, which could further reduce inference time.

11 Inference on Intel i7-1355U

Model 2’s inference on the target CPU averaged 0.1208 seconds per question across five validation examples (e.g., predicting “denver broncos” for “Which NFL team represented the AFC at Super Bowl 50?”). BERT’s inference time (0.30 seconds) underscores DistilBERT’s efficiency for real-time applications.

12 Conclusion

This study presents an efficient DistilBERT-based QA model for SQuAD v1.1, achieving an F1 score of 0.6536 and an inference time of 0.1208 seconds on a 13th Gen Intel i7-1355U CPU. Through EDA, data augmentation, and hyperparameter optimization, we demonstrate a practical solution for resource-constrained environments. Future work could explore quantization, pruning, or larger datasets to enhance performance while maintaining efficiency.

A Code Listings

A.1 Data Preprocessing and Augmentation

Listing 1: Data Preprocessing and Augmentation

```
1 from transformers import DistilBertTokenizer
2 from nltk.corpus import wordnet
3 import tensorflow as tf
4
```

```

5 tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-
uncased')
6
7 def paraphrase_question(question):
8     words = question.split()
9     new_words = []
10    for word in words:
11        syns = wordnet.synsets(word)
12        if syns and word in ['what', 'who']:
13            new_word = syns[0].lemmas()[0].name() if syns[0].lemmas()
else word
14            new_words.append(new_word)
15        else:
16            new_words.append(word)
17    return ' '.join(new_words)
18
19 def preprocess_data(examples, max_length=384):
20     inputs = tokenizer(
21         examples['question'], examples['context'],
22         max_length=max_length, truncation=True, padding='max_length',
23         return_tensors='tf'
24     )
25     start_positions = tf.constant([example['answer_start'] for example
in examples])
26     end_positions = tf.constant([example['answer_end'] for example in
examples])
27     return inputs, start_positions, end_positions

```

A.2 Model Training (Model 2)

Listing 2: Training Code for Model 2

```

1 from transformers import TFDistilBertForQuestionAnswering,
create_optimizer
2 import tensorflow as tf
3
4 model = TFDistilBertForQuestionAnswering.from_pretrained('distilbert-
base-uncased')
5 num_train_steps = 45000 // 8 * 5
6 optimizer, lr_schedule = create_optimizer(
7     init_lr=2e-5, num_train_steps=num_train_steps, num_warmup_steps=500
8 )
9
10 model.compile(
11     optimizer=optimizer,
12     loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True
, label_smoothing=0.05),
13     metrics=['accuracy']
14 )
15
16 train_dataset = tf.data.Dataset.from_tensor_slices((inputs, {'
start_positions': start_positions, 'end_positions': end_positions}))
17 train_dataset = train_dataset.shuffle(1000).batch(8)
18 model.fit(train_dataset, epochs=5, callbacks=[tf.keras.callbacks.
EarlyStopping(patience=3)])

```

A.3 Inference Implementation

Listing 3: Inference Code for Model 2

```
1 import tensorflow as tf
2 from transformers import AutoTokenizer,
   TFDistilBertForQuestionAnswering
3 import time
4
5 model_path = "best_distilbert_model"
6 tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
7 loaded_model = TFDistilBertForQuestionAnswering.from_pretrained("
   distilbert-base-uncased")
8 try:
9     loaded_model.load_weights(model_path)
10 except Exception as e:
11     print(f"Warning: Could not load weights from {model_path}. Error: {
        e}")
12
13 def predict_answer_distilbert(question, context, model, tokenizer):
14     inputs = tokenizer(question, context, max_length=512, truncation="
        only_second", padding="max_length", return_tensors="tf")
15     outputs = model(inputs)
16     start_logits = outputs.start_logits[0]
17     end_logits = outputs.end_logits[0]
18     start_pred = tf.argmax(start_logits).numpy()
19     end_pred = tf.argmax(end_logits).numpy()
20     input_ids = inputs["input_ids"].numpy()[0]
21     sep_idx = -1
22     for i, id in enumerate(input_ids):
23         if id == tokenizer.sep_token_id:
24             sep_idx = i
25             break
26     if sep_idx != -1 and start_pred <= end_pred and start_pred > 0 and
        end_pred < sep_idx:
27         answer_tokens = input_ids[start_pred:end_pred+1]
28         answer = tokenizer.decode(answer_tokens, skip_special_tokens=
            True)
29         return answer
30     else:
31         return "No answer found"
```

References

- Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. Electra: Pre-training text encoders as discriminators rather than generators, 2020. URL <https://arxiv.org/abs/2003.10555>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019. URL <https://arxiv.org/abs/1810.04805>.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2020. URL <https://arxiv.org/abs/1909.11942>.

- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019. URL <https://arxiv.org/abs/1907.11692>.
- Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one?, 2019. URL <https://arxiv.org/abs/1905.10650>.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023. URL <https://arxiv.org/abs/1910.10683>.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text, 2016. URL <https://arxiv.org/abs/1606.05250>.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, 2020. URL <https://arxiv.org/abs/1910.01108>.
- Siqi Shen, Zhequan Dong, Jianda Ye, Linjian Ma, Zhe Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. Q-bert: Hessian based ultra low precision quantization of bert. *arXiv preprint arXiv:1909.05840*, 2021.
- Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. Mobilebert: a compact task-agnostic bert for resource-limited devices, 2020. URL <https://arxiv.org/abs/2004.02984>.
- Jason Wei and Kai Zou. Eda: Easy data augmentation techniques for boosting performance on text classification tasks, 2019. URL <https://arxiv.org/abs/1901.11196>.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Huggingface’s transformers: State-of-the-art natural language processing, 2020. URL <https://arxiv.org/abs/1910.03771>.
- Wei Yang, Yuqing Xie, Aileen Lin, Xingyu Li, Luchen Tan, Kun Xiong, Ming Li, and Jimmy Lin. End-to-end open-domain question answering with. In *Proceedings of the 2019 Conference of the North*. Association for Computational Linguistics, 2019. doi: 10.18653/v1/n19-4013. URL <http://dx.doi.org/10.18653/v1/N19-4013>.
- Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. Xlnet: Generalized autoregressive pretraining for language understanding, 2020. URL <https://arxiv.org/abs/1906.08237>.
- Ofir Zafrir, Guy Boudoukh, Peter Izsak, and Moshe Wasserblat. Q8bert: Quantized 8bit bert. In *2019 Fifth Workshop on Energy Efficient Machine Learning and Cognitive Computing - NeurIPS Edition (EMC2-NIPS)*, page 36–39. IEEE, December 2019. doi: 10.1109/emc2-nips53020.2019.00016. URL <http://dx.doi.org/10.1109/EMC2-NIPS53020.2019.00016>.